Dockerizing the Library Management System

A step-by-step guide to installing Docker and running your Django + PostgreSQL + Celery project in containers

---

1. Install Docker

Your shell prompt (`hari@DESKTOP-M7K2RJM`) shows you're working inside WSL (Windows Subsystem for Linux). You have two ways to get Docker working there:

Option A — Docker Desktop with WSL2 integration (recommended)

- Download and install **Docker Desktop for Windows** from docker.com.
- During setup, make sure **"Use WSL 2 based engine"** is enabled.
- Open Docker Desktop → Settings → Resources → WSL Integration → enable integration for your Ubuntu distro.
- Restart your WSL terminal, then verify from inside WSL:

```
docker --version
docker compose version
```

Both commands should print version numbers with no errors.

Option B — Docker Engine directly inside WSL (no Docker Desktop)

```
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
sudo usermod -aG docker $USER
# log out and back into WSL, then:
sudo service docker start
docker --version
```

This avoids Docker Desktop entirely but you'll need to start the Docker daemon manually each time WSL restarts (`sudo service docker start`).

2. Why requirements.txt was failing

Before touching Docker again, it's worth knowing that Docker doesn't fix a broken requirements.txt by itself — it just moves the same pip install into a Linux container. The most common causes of pip failures for a Django + PostgreSQL + Celery stack are:

- **psycopg2** (not psycopg2-binary) needs system build tools (gcc, libpq-dev) that aren't present in a plain image.
- Pinned versions that no longer exist on PyPI (packages get yanked or renamed).

- Windows-only packages accidentally pinned, e.g. `pywin32` or `pypiwin32`, which simply don't exist on Linux.
- A stale `pip` that can't resolve newer dependency metadata.

Your current Dockerfile works around this with `pip install ... || true`, which hides the error instead of fixing it — the build "succeeds" but some packages may silently be missing, and the app can crash later with an ImportError. Section 3 replaces this with a proper fix.

3. A corrected Dockerfile

This installs the Linux build dependencies Postgres drivers need, upgrades pip first, and lets install failures surface immediately (fail fast) instead of being swallowed.

```
FROM python:3.11-slim

WORKDIR /app

# System packages needed to build psycopg2 / Pillow / other C-extension deps
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential \
    libpq-dev \
    gcc \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir --upgrade pip && \
    pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["python", "manage.py", "runserver", "0.0.0.0:8000"]
```

If `docker compose build` now fails, the error will name the exact package that's broken — fix or remove that one line in requirements.txt rather than silencing all errors.

4. A complete docker-compose.yml

Your project defines `library_system/celery.py`, and the static files include a Redis/Celery guide — so this setup adds a Redis broker and a Celery worker service alongside your existing web and db services. It also adds a database healthcheck so the web container waits for Postgres to actually be ready, not just "started".

```
services:
  db:
    image: postgres:15
    restart: always
    environment:
      POSTGRES_DB: library_db
      POSTGRES_USER: library_user
      POSTGRES_PASSWORD: library_pass
    volumes:
      - postgres_data:/var/lib/postgresql/data
```

```
    ports:
      - "5432:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U library_user -d library_db"]
      interval: 5s
      timeout: 5s
      retries: 5

  redis:
    image: redis:7
    restart: always
    ports:
      - "6379:6379"

  web:
    build: .
    command: python manage.py runserver 0.0.0.0:8000
    volumes:
      - .:/app
    ports:
      - "8000:8000"
    depends_on:
      db:
        condition: service_healthy
      redis:
        condition: service_started
    environment:
      DB_NAME: library_db
      DB_USER: library_user
      DB_PASSWORD: library_pass
      DB_HOST: db
      DB_PORT: 5432
      REDIS_HOST: redis
      REDIS_PORT: 6379

  celery:
    build: .
    command: celery -A library_system worker -l info
    volumes:
      - .:/app
    depends_on:
      - web
      - redis
    environment:
      DB_NAME: library_db
      DB_USER: library_user
      DB_PASSWORD: library_pass
      DB_HOST: db
      DB_PORT: 5432
      REDIS_HOST: redis
      REDIS_PORT: 6379

volumes:
  postgres_data:
```

If you don't use Celery yet, skip the redis and celery services and keep the original two-service file.

5. Check settings.py reads these variables

Compose only sets environment variables inside the container — Django still needs to read them. Confirm `library_system/settings.py` has something like this:

```
import os

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': os.environ.get('DB_NAME', 'library_db'),
        'USER': os.environ.get('DB_USER', 'library_user'),
        'PASSWORD': os.environ.get('DB_PASSWORD', 'library_pass'),
        'HOST': os.environ.get('DB_HOST', 'localhost'),
        'PORT': os.environ.get('DB_PORT', '5432'),
    }
}

CELERY_BROKER_URL = os.environ.get(
    'CELERY_BROKER_URL',
    f"redis://{os.environ.get('REDIS_HOST', 'localhost')}:{os.environ.get('REDIS_PORT', '6379')}/0
"
)
```

If settings.py currently hardcodes `HOST: 'localhost'`, that's a separate bug worth fixing — inside Docker, "localhost" refers to the web container itself, not the db container, so the app can't reach Postgres until this points at the `db` service name instead.

6. Build and run the project

From the project root (where docker-compose.yml lives):

```
cd ~/library_system/library-management-system

# Build images (rebuild any time Dockerfile or requirements.txt changes)
docker compose build

# Start everything in the background
docker compose up -d

# Watch logs
docker compose logs -f web
```

Then run one-off Django management commands inside the running web container:

```
docker compose exec web python manage.py migrate
docker compose exec web python manage.py createsuperuser
docker compose exec web python manage.py collectstatic --noinput
```

Visit the app at:

```
http://localhost:8000
```

Everyday commands

Command	What it does
`docker compose up -d`	Start all containers in the background
`docker compose down`	Stop and remove containers (keeps the DB volume)
`docker compose down -v`	Stop containers and delete the DB volume too
`docker compose ps`	List running containers
`docker compose logs -f web`	Follow logs for the web container
`docker compose exec web bash`	Open a shell inside the web container
`docker compose build --no-cache`	Rebuild ignoring the Docker layer cache

7. Troubleshooting checklist

- **Build fails on a specific package** — remove the version pin for that line in requirements.txt (let pip pick a compatible version) or replace `psycopg2` with `psycopg2-binary`.
- **"connection refused" to the database** — almost always means settings.py still points at `localhost` instead of the `db` service name, or the web container started before Postgres was ready (the healthcheck in Section 4 fixes the second case).

- **Port 8000 or 5432 already in use** — another process on Windows or WSL is bound to it; stop it, or change the left-hand side of the port mapping, e.g. `"8001:8000"`.
- **Code changes don't show up** — confirm the `volumes: - .:/app` line is present so your local folder is mounted live into the container.
- **Static files missing (CSS not loading)** — run `collectstatic` (Section 6) and confirm STATIC_ROOT is set in settings.py.
- **Slow builds** — the apt-get and pip layers only re-run when requirements.txt or the Dockerfile change, thanks to Docker's layer cache, so keep `COPY requirements.txt .` before `COPY . .` as shown in Section 3.

If a build still fails after applying these fixes, share the exact error text and the contents of requirements.txt for a targeted fix.